15,471,951 members

Sign in

articles Q&A forums stuff lounge ?

Search for articles, questions,

A Revolutionary New Approach to Custom Drawn Menus



.dan.g.

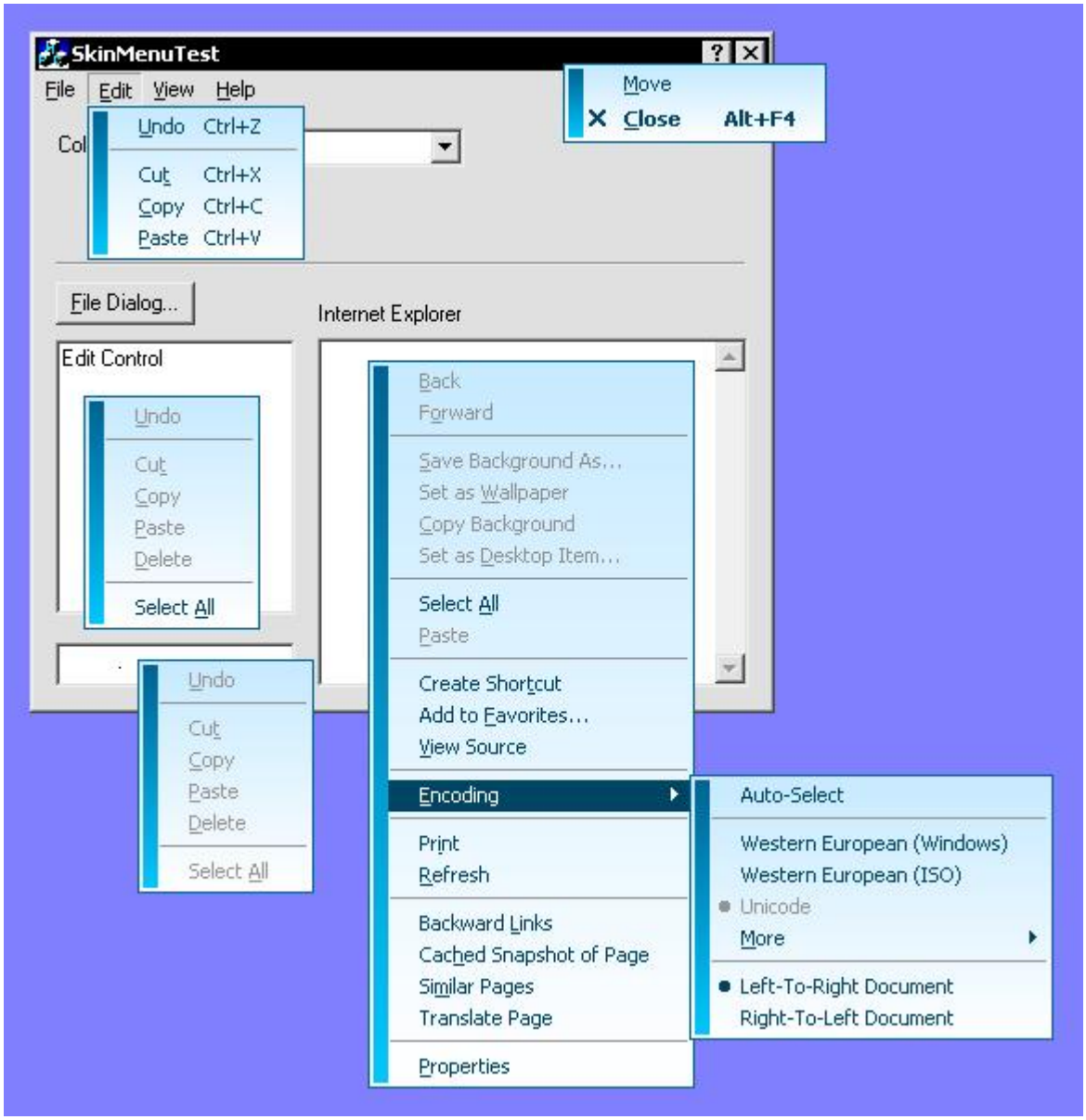
26 May 2003

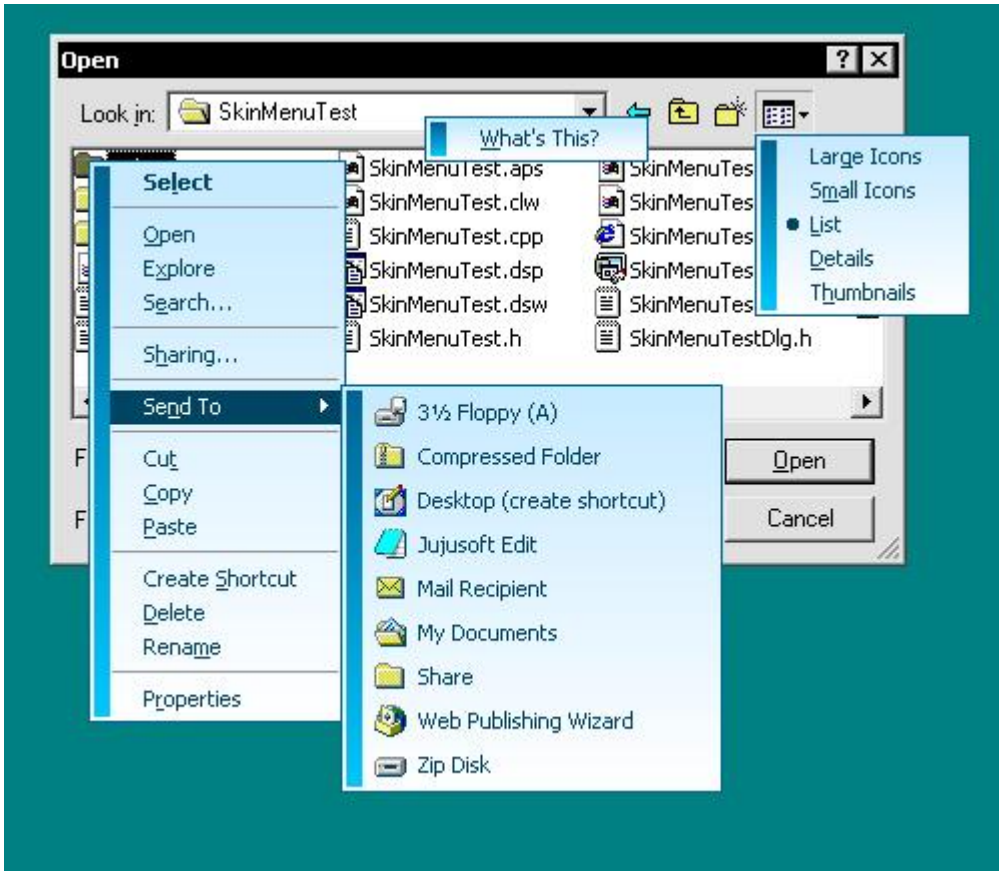
Rate me: 4.95/5 (117 votes)

A new and unique approach to the perennial problem of how to change the default appearance of Windows menus

Download source files - 51 Kb

Download demo executable - 102 Kb


SkinMenuTest
File
Edit
View
Help
Undo Ctrl+Z
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Move
Close Alt+F4
File Dialog...
Internet Explorer
Edit Control
Undo
Cut
Copy
Paste
Delete
Select All
Back
Forward
Save Background As...
Set as Wallpaper
Copy Background
Set as Desktop Item...
Select All
Paste
Create Shortcut
Add to Favorites...
View Source
Encoding
Print
Refresh
Backward Links
Cached Snapshot of Page
Similar Pages
Translate Page
Properties
Auto-Select
Western European (Windows)
Western European (ISO)
Unicode
More
Left-To-Right Document
Right-To-Left Document



Rev 1.2

Introduction

Changing the default appearance of elements of the Windows GUI has been a never ending challenge for Windows programmers since the first release of the Windows API. And anyone who has seriously tried to change the appearance of menus (beyond simple owner-draw techniques) will know that it represents one of holy grails of Windows GUI programming.

Note: To reduce the clutter in the article I am going to use the word 'skin' in place of 'changing the default appearance'.

What I want to present here is an entirely **new and original** approach to skinning menus which will allow you to skin **every** menu that your application produces, including (but not restricted to):

- menubars (e.g.. application main menu)
- all right-click (context) popup menus (including your own and those of internet explorer and the standard file dialog)
- the system menu (both in a window's title bar and on the Taskbar)
- toolbar drop-down menus
- your own or 3rd party owner-drawn menus (check-out the 'New' and 'Send To' sub menus in the File Dialog)

The Challenge

Windows menus have always been a black box to developers, with all of the implementation hidden away inside *user32.dll*. All that Windows has allowed a developer to do is to define the menu as owner-draw and then handle the `WM_MEASUREITEM` and `WM_DRAWITEM` messages for drawing one or more menu items.

This is fine if your interest in owner-draw menus is to replace the standard text interface of, say, a 'select color' menu with menu items displaying the colors themselves, but it falls way short of an architecture to replace **all** menus with a different style to the gray (or `COLOR_MENU`) default.

In particular:

- It would require you to have access to every menu handle in your application so that you could modify them to be owner-draw (as an experiment: try getting a handle to the context menu of an edit box)
- It would require you to be able to insert code into every window or control that displayed the menu to handle the `WM_MEASUREITEM` and `WM_DRAWITEM` messages
- It provides no mechanism for replacing the non-client (border) drawing with your own

Much more fundamentally however, there is no way for you to handle menus that someone else (like Windows itself) has already defined as owner-draw, because generally you will have no idea what memory structures have been used for storing the information needed for the drawing stage.

And it is this last point which ultimately kills off owner-draw as a possible solution to the problem.

The Solution

Part 1

How the 'solution' occurred to me, I have no idea. I may have been taking a bath, or possibly sitting under a tree when an apple fell on my head, I don't know. But it did. 'What if', I thought, 'I could redirect all of the painting of a menu away from the screen and onto a memory DC so that i could post-process it. 'Then I could do anything I liked to it before finally rendering the menu on the screen.' And the rest as they say, was hard slog.

Part 2

The implementation of this solution can be broken down into 2 distinct tasks which needed solving:

- A means of detecting when a menu is about to be shown.
- A way of overriding the drawing for a particular menu.

The solution to the former was definitely the easier of the two. All I needed to do was to install a Windows Hook to detect the creation of all menu windows.

Note: For those of you who don't know, menu windows have a special class name (`#32768`) which makes their detection no more difficult that any other window.

Having caught the menu window just before its creation, however, still leaves the question, 'What to do with it?' to which the answer is (of course) subclassing. And this is where I must give due thanks to Paul DiLascia and his now legendary `CSubclassWnd` implementation. `CSubclassWnd` is an MFC class that Paul wrote to allow the interception and overriding of all Windows messages for a given window, even when you did not create the window itself. So that's essentially it: Catch all menu windows just before they are shown and subclass them so that we can override the drawing.

Overriding the Default Drawing

Its often the case in programming (and certainly the case in engineering) that if the design is sound then the implementation will generally fall into place. Unfortunately, this was not exactly one of those occasions. I've found with Windows GUI programming that there are just too many quirks and variations between the various flavours of Windows (95 -> XP) not to have to resort to using undocumented tweaks and fudge factors to achieve a reasonable solution.

Moreover, in this particular situation, there was **no** documentation to either support or suggest that I stood any reasonable chance of being successful. My first task was to determine what Windows messages prompted menus to (re)draw themselves so that I could replace these with my custom implementations. Fortunately, what I found was that menu drawing is quite simple (if undocumented). This is a summary of the Windows messages which needed to be handled:

- `WM_PRINT` - requests the menu to draw its non-client and/or client areas in the DC supplied in `wParam` (Windows 9x, 2K, XP)
- `WM_PRINTCLIENT` - requests the menu to draw its client area in the DC supplied in `wParam` (Windows 9x, 2K, XP)
- `WM_PAINT` - requests the menu to draw the foreground of its invalid area (Windows 9x)
- `WM_ERASEBKGND` - requests the menu to draw the background of its invalid area (Windows 9x)
- `0x01e5` (undocumented) - requests the menu to redraw the item supplied in the message `wParam` using an internal method (Windows 9x, 2K, XP)

This last message (`0x01e5`) is the most interesting and the most crucial discovery I made on this project. Windows sends it to the menu every time you twitch the mouse inside or outside the menu, so that the menu can redraw the specified item. At first it might seem like a gross inefficiency since no other window controls behave that way, but if you've got menu animation turned on you'll see why its necessary.

Normally when GUI events happen in Windows its usual to wait until they are complete and then redraw the control as required. Not so with menu fading; in this case, the menu fading is carried out

after the event has happened using a system timer (I believe). So if you only redraw the menu once after the event triggers then you get drawing artifacts all over the place. Luckily, it was simply a matter of calling `SetRedraw()` before and after the default implementation of this message to ensure that the menu did not do its 'under the cover' drawing, followed by invalidating the rectangle of the menu item in question.

Replacing the System Colors

Once I had control of the redrawing, I still needed to work out how to replace the system colors with the users choices.

As I had already been working extensively on a application skinning system and had been using `TransparentBlt()` to good effect there, I hit on the idea of using it to do multi-pass post-processing of the menu image, replacing one system color in each pass.

I had anticipated a fair performance hit with this idea but in practice it seems to be quite reasonable. Even under Windows 98 where I have substituted my own implementation of `TransparentBlt()` (the default version has a well documented resource leak) it still performs quite acceptably, although I have not done nearly enough testing on lower end machines.

I have also gone to some lengths to ensure that I carry out the minimum number of `TransparentBlt()` in thsoe case where some system colors resolve to the same `COLORREF`.

Other Implementation Details

Some of you sharper readers may have noticed an implicit reference to a menu handle (`HMENU`) in the previous paragraph ('...rectangle of the menu item...') which I had not mentioned before. Whilst its true that the menu skinning can be achieved without having the menu handle, the implementation can be made more efficient by only invalidating the menu items which need redrawing rather than the whole window.

Determining the menu handle, however, is a real pain because Windows provides no explicit mapping between a menu window and its associated menu handle. This is where the fudge I referred to earlier comes in. To retrieve the menu handle, I wait until I detect a `WM_INITMENUPOPUP` and then either attach it to the menu window if it has already been created, or hold on to it until the menu window is subclassed and add it then. There's no guarantee that the match-up will correct but it seems to work okay. A real fudge but without an alternative as far as I can see.

Using the Code

- Add the following source files to your project:

Note: in my demo project, these files are in a separate 'skinwindows' folder because they form a subset of a much larger skinning system, but there is no need for you to do the same.

  - `CHookMgr` (*hookmgr.h*) - template class for simplifying hooking
  - `CSkinBase` (*skinbase.h/.cpp*) - some hard core helper methods
  - `CSkinGlobals` (*skinglobals.h/.cpp, skinglobalsdata.h*) - helper class for overriding default Windows colors
  - `CSkinMenu` (*skinmenu.h/.cpp*) - menu window overriding
  - `CSkinMenuMgr` (*skinmenumgr.h/.cpp*) - menu window hooking and management
  - `CSubclassWnd` (*subclass.h/.cpp*) - subclassing helper class (heavily modified from Paul DiLascia's original)
  - `CWinClasses` (*winclasses.h/.cpp*) - helper class for retrieving and testing window classes
  - *wclassdefines.h* - convenient `#defines` for all window classes (and some others)
  - *skincolors.h* - color mappings

- Add `NO_SKIN_INI` to the preprocessor definitions in your project settings. This is to avoid compilation problems due to missing files, because this project forms part of a larger system which supports loading color information from a file, which is not included here.
- Initialize the skin menu manager in your `CWinApp` derived application `InitInstance()` method as follows:

```
#include "skinmenumgr.h"
                          // assumes files are in same folder
                          // as rest of the project

BOOL CMyApp::InitInstance()
{
       :
       :
    CSkinMenuMgr::Initialize();
       :
       :
}
```

Have a look at the implementation of `CSkinMenuMgr::Initialize()` for more detail on the options available. In particular you can elect to display a sidebar of a given width and give the menu border a flat or beveled edge.

For total control over how the menus are drawn, including the menu background, derive a class from `ISkinMenuRender` which is defined in *skinmenu.h* and pass to `CSkinMenu::SetRenderer()` which is a static method. Then during the drawing process, the `CSkinMenu` class will call back into your derived class giving you the opportunity to drawn whichever portions of the menu you choose to. (see `CSkinMenuTestDlg` for an example implementation)

Further Work

- It doesn't handle scrolling menus (thanks to saltynuts2002)
- It doesn't work under Windows CE (thanks to João Paulo Figueira)
- It's a bit slow under XP.
- The rendering speed could probably be improved generally by taking more account of the clip box.

Warnings

- When using keyboard navigation under the debugger be prepared for an internal Windows breakpoint to show up; something that does seem to be a problem outside the debugger.
- To put it bluntly, debugging under Windows 98 and Me is to be avoided if at all possible. Windows does not like having menus skinned on these platforms and you should expect regular reboots if you set breakpoints inside the menu redrawing code.
- Although the demo application runs fine on Windows 98 and Me, the more fragile nature of these platforms means that the slightest bugs can crash the system (not Blue-Screen but you still have to restart)
- When carrying out non-client drawing it is imperative to restore the state of the DC before you finish. It appears that Windows re-uses the same DC for rendering all menus, and once you've messed with it it stays messed.
- It's a bit slow under XP for reasons I'm looking into.
- Please take the time to read the code in detail before asking any questions about it.

Copyright

The code is supplied here for you to use and abuse without restriction, except that you may not modify it and pass it off as your own. The concept and design, however, remains my intellectual property in perpetuity.

History

- **1.0** Initial Release
- **1.1** Bug Fixes
    - added support for keyboard navigation, including multi-column menus
    - corrected color substitution for disabled items and dividers
    - added support for unskinning when menus are hidden (Win 9x). This fixes what Windows reported as a resource leak with menu bars.
- **1.2** Bug Fixes + Features
    - updated to support CHookMgr
    - handles scrolling menus much better
    - disables the client area callback under 95/NT4 but still supports non-client drawing

- adds an option to CSkinMenuMgr::Initialize() to disable menu skinning under XP

License

This article has no explicit license attached to it but may contain usage terms in the article text or the download files themselves. If in doubt please contact the author via the discussion board below.

A list of licenses authors might use can be found here

Written By

.dan.g.

Software Developer Maptek

Australia

.dan.g. is a naturalised Australian and has been developing commercial windows software since 1998.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

why my listbox hooker works ilnormal?
joshua0137 11-Aug-12 5:36

Re: why my listbox hooker works ilnormal?
joshua0137 12-Aug-12 1:45

I vote 5
joshua0137 11-Nov-11 1:03

Problem with Windows XP x64
YoSilver 2-Aug-11 7:36

My vote of 5
xubin 15-Dec-10 19:24

About SkinMenuTest question
PUBTEAM 26-Sep-09 19:36

Re: About SkinMenuTest question
wanglinhai888 16-Dec-09 12:07

tank you
mrshcom 17-Sep-09 8:25

Demo project NOT works under Vista?!
y_yy2008 29-Jan-09 14:55

Re: Demo project NOT works under Vista?!
phoaivu 29-Jun-09 21:50

Re: Demo project NOT works under Vista?!
phoaivu 13-Jul-09 23:43

CsystemTray
yyaw 17-Jul-08 18:42

clear
hallouf 1-Mar-08 8:44

"Unknown" messages
Sega-Zero 9-Sep-07 23:32

Changing Menu Font at run-time ???
ana_v123 5-Apr-06 21:13

Good job!!!
thirdwolf 20-Mar-06 19:28

color consistency
T1TAN 29-Aug-05 0:45

How to reduce flicker in the overlap area with a parent menu
ririye 24-Apr-05 17:00

Wonderful job!
ririye 22-Apr-05 15:55

About "Modifying the highlighting on a menu item?"
younger_dl 4-Apr-05 16:58

Re: About "Modifying the highlighting on a menu item?"

younger_dl 4-Apr-05 20:21

Re: About "Modifying the highlighting on a menu item?"
Tim Stubbs 4-Apr-05 22:09

Re: About "Modifying the highlighting on a menu item?"
younger_dl 5-Apr-05 2:44

Re: About "Modifying the highlighting on a menu item?"
Tim Stubbs 5-Apr-05 2:50

Re: About "Modifying the highlighting on a menu item?"
younger_dl 5-Apr-05 3:07

Refresh

1 2 3 4 5 6 7 8 9 Next ⊳

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2003 by .dan.g.
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1